Summarize

Chapter 16. The Evolutionary Architect

As we have seen so far, microservices give us a lot of choices, and accordingly a lot of decisions to make. For example, how many different technologies should we use, should we let different teams use different programming idioms, and should we split or merge a microservice? How do we go about making these decisions? With the faster pace of change and the more fluid environment that these architectures allow, the role of the architect also has to change. In this chapter, I'll take a fairly opinionated view of what the role of an architect is and hopefully launch one final assault on the ivory tower.

What's in a Name?

> *You keep using that word. I do not think it means what you think it means.*
>
> —Inigo Montoya, from *The Princess Bride*

Architects have an important job. They are in charge of making sure the system has a joined-up technical vision, one that should help deliver the software that customers need. In some places, they may have to work with only one team, in which case the role of the architect and that of the technical lead are often one and the same. In other places, they may be defining the vision for an entire program of work, coordinating with multiple teams across the world, or perhaps even an entire organization. At whatever level architects operate, their role is a tricky one to pin down, and despite it often being the obvious career progression for developers in enterprise organizations, it is also a role that gets more criticism than virtually any other in our field. More than any other role, architects can have a direct impact on the quality of the systems built, on the working conditions of their colleagues, and on their organization's ability to respond to change, and yet their role seems very poorly understood. Why is that?

Our industry is a young one. We seem to forget sometimes that we have been creating programs that run on what we recognize as computers for only 75 years or so. Our profession doesn't fit into a nice neat box that society as a whole understands. We aren't

like electricians, plumbers, medical doctors, or engineers. How many times have you told someone what you do at a party, only for the conversation to stop? The world as a whole struggles to understand software development—as I've outlined multiple times throughout this book, we frequently don't seem to understand it ourselves.

So we borrow from other professions. We call ourselves software "engineers," or "architects." But we aren't architects or engineers in the way that society understands those professions. Architects and engineers have a rigor and discipline we could only dream of, and their importance in society is well understood. I remember talking to a friend of mine the day before he became a qualified architect. "Tomorrow," he said, "if I give you advice down at the pub about how to build something and it's wrong, I get held to account. I could get sued, as in the eyes of the law I am now a qualified architect and I should be held responsible if I get it wrong." The importance of these jobs to society means that there are required qualifications people have to meet. In the UK, for example, a minimum of seven years' study is required before you can be called an architect. But these jobs are also based on a body of knowledge going back thousands of years. And software architects? Not quite. Which is partly why I view many forms of IT certification as worthless, as we know so little about what "good" looks like.

I don't say this to belittle the term *software engineering*,[1] coined back in the 1960s by Margaret Hamilton, but it was as much aspirational as it was about the current reality. The term emerged as a call to improve the quality of the software being created, and in recognition of the fact that software projects often failed and yet were increasingly being used in vital mission- and safety-critical fields. Much work has been done to improve the situation since then, but my own take after 20 years in the industry is that we've still got a lot to learn about doing a good (or at least a better) job.

Part of us wants recognition, so we borrow names from other professions that already have the recognition we crave. But this can prove problematic, if we borrow *working practices* from those professions without understanding the mindset behind them or taking into account how software development is different from, say, civil engineering. None of this should be taken as an argument that we shouldn't aim to have more rigor in our work—just that we cannot simply borrow ideas from elsewhere and assume they will work for us. Our industry is very young, and the challenge is that we have far fewer absolutes around which we agree as an industry.

Perhaps the term *architect*, or at least the common understanding of what architects do, has done the most harm in this way: the idea of someone who draws up detailed plans for others to interpret and expects them to be carried out; the balance of part artist, part engineer, overseeing the creation of what is often a singular vision, with all other view-

points being subservient, except for the occasional objection from the structural engineer regarding the laws of physics. In our industry, this view of the architect leads to some terrible practices, with architects creating diagram after diagram, page after page of documentation, with a view to inform the construction of the perfect system, while failing to take into account the fundamentally unknowable future, and utterly devoid of any understanding of how hard their plans will be to implement, or whether or not they will actually work, let alone having any ability to change as we learn more.

But architects of the built environment are operating in a different realm from that of software architects. Their constraints are different, the end product different. The cost of change is so much higher in construction than it is in software development. You can't unpour concrete, but you can change code, and even the infrastructure we run our code on is much more malleable than before, thanks to virtualization. Buildings are fairly fixed once built—they can be changed, or expanded, or torn down, but the associated costs are very high. But we expect our software to continually change to suit our needs.

So if software architecture is different from the architecture of the built environment, perhaps we should be a bit clearer in terms of what software architecture actually is.

What Is Software Architecture?

One of the most famous definitions of software architecture comes via an email from Ralph Johnson: "Architecture is about the important stuff. Whatever that is."[2] So does this mean that anything important is done by the architect? Does that mean that all other work being done is *not* important? The issue with this oft-quoted statement is that it's often used in isolation, without any understanding of the wider response in which Ralph shared it. Firstly, it's clear that he is talking from the perspective of a software developer. He goes on to say:

> *So, a better definition would be "In most successful software projects, the expert developers working on that project have a shared understanding of the system design. This shared understanding is called 'architecture.' This understanding includes how the system is divided into components and how the components interact through interfaces. These components are usually composed of smaller components, but the architecture only includes the components and interfaces that are understood by all the developers."*
>
> *This would be a better definition because it makes clear that architecture is a social construct (well, software is too, but architecture is even more so) because it doesn't just depend on the software, but on what part of the software is considered important by group consensus.*

Here Ralph is using the term *components* in its most general sense. In the context of this book, we can think of the components as our microservices, and perhaps the modules inside those microservices.

Software architecture is the shape of the system. Architecture happens, by design or accident. We make a series of ad hoc decisions, and we end up with the results—without thinking about things in terms of architecture, we end up with architecture nonetheless. Architecture can sometimes be what happens while we're busy making other plans.

A dedicated architect is someone who should see and understand that whole system, understand the forces acting on it. They need to ensure there is a vision for the architecture that is fit for purpose and is clearly understood—an architectural vision that satisfies the needs of the system and its users, as well as those of the people who work on the system itself. Looking at only one facet—e.g., logical but not physical, shape but not developer experience—limits an architect's effectiveness. If you accept that architecture is about understanding the system, then limiting the scope of what you care about limits your ability to reason and make changes.

Architecture can be invisible to the people living with it. It can be so slight as to not really be there. It can be something that guides and helps achieve the right outcome. It can be suffocating and overbearing. It can delight without you realizing it is even a thing, and crush the spirit from you without any malice being intended. So whether or not architecture is "about the important stuff," it's certainly *important*.

Another pithy quote that is often used to define software architecture comes from the same article where Martin shares Ralph's views: "So you might end up defining architecture as *things that people perceive as hard to change.*" Martin's idea that architecture is

the stuff that's hard to change makes sense at some level and brings us back to the concept of architecture in the built environment. Where things are harder to change, they need a bit more up-front thought to really make sure we are going in the right direction. But there is a problem with taking a simple definition of a complex idea and running with that as a working definition—if this statement were entirely how you thought about software architecture, you'd miss out on a lot. Yes, a lot of software architecture is about thinking about the things that will be hard to change, but it is also about creating space to allow change in the design.

Making Change Possible

Coming back to the world of buildings rather than software systems: architect Mies van der Rohe arguably did more to pioneer what we now think of as the modern skyscraper than any other architect—his famous Seagram Building became the blueprint for much of what followed. The Seagram Building differs from a lot of what came before. The outer walls of the building are nonstructural—they wrap a steel outer frame. The main building services—lifts,[3] stairways, air conditioning, water and waste, and the electrical system—run through a central concrete core. Watch a modern high-rise being constructed today, and it's this central concrete core that is built first, a giant crane often seen perching on top. Each floor of the Seagram Building has no interior structural walls—this means that you have total flexibility in terms of how the space is used. You can reconfigure the space as you see fit, routing electrical wiring and air conditioning to different parts of each floor via suspended ceilings and ducts in the floor itself.

It's interesting to note that the Seagram Building was developed using a process in which the design of the building evolved while the construction was carried out. Now where have we seen that idea before?

The idea with this design was to deliver what Mies van der Rohe called "universal space"—a large, single-span volume that could be reconfigured to suit different needs. The use of buildings changes, so the idea was to deliver space that is as flexible as possible in terms of how it can be used. In this way, Mies van der Rohe not only had to focus on the fundamental aesthetics of the building, finding a space for core services that would be difficult if not impossible to change later, but he also had to ensure that the building could be used in different ways than originally envisaged. Shortly, we'll look at how we allow for change in the space of a microservice architecture.

An Evolutionary Vision for the Architect

Our requirements as architects of software shift more rapidly than they do for people who design and build buildings—as do the tools and techniques at our disposal. The things we create are not fixed points in time. Once launched into production, our software will continue to evolve as the way it is used changes. For most things we create, we have to accept that once the software gets into the hands of our users, we will have to react and adapt rather than expect a never-changing artifact. Thus, software architects need to shift their thinking away from creating the perfect end product and focus instead on helping create a *framework* in which the right systems can emerge and continue to grow as we learn more.

Although I have spent much of the chapter discouraging comparisons to other professions, there is an analogy that I like when it comes to the role of the IT architect and that I think encapsulates this aspect of the role well. Erik Doernenburg at Thoughtworks first shared with me the idea that we should think of the architect's role more as town planner than architect of the built environment. The role of the town planner should be familiar to any of you who have played SimCity or Cities: Skylines before. A town planner's role is to look at a multitude of sources of information and then attempt to optimize the layout of a city to best suit the needs of present-day citizens, while also taking into account future use. The way they influence how the city evolves, though, is interesting. They do not say, "Build this specific building there"; instead, they define zones that allow for local decision making within certain constraints. So, as in SimCity, you might designate part of your city as an industrial zone, and another part as a residential zone. It is then up to other people to decide what buildings get created, but there are restrictions: if you want to build a factory, it will need to be in an industrial zone. Rather than worrying too much about what happens in one zone, the town planner will instead spend far more time working out how people and utilities move from one zone to another.

More than one person has likened a city to a living creature. The city changes over time. It shifts and evolves as its occupants use it in different ways, or as external forces shape it. The town planner does their best to anticipate these changes but accepts that trying to exert direct control over all aspects of what happens is futile. So our architects as town planners need to set direction in broad strokes and get involved in being highly specific about implementation detail only in limited cases. They need to ensure that the system is fit for purpose now but also a platform for the future.

The comparison with software should be obvious. As our users use our software, we need to react and change. We cannot foresee everything that will happen, and so rather

than plan for each and every eventuality, we should plan to allow for change by avoiding the urge to overspecify every last thing. Our city—the system—needs to be a good, happy place for everyone who uses it.

Defining System Boundaries

To continue with the metaphor of the architect as town planner for a moment, what are our zones? These are our microservice boundaries, or perhaps coarse-grained groups of microservices. As architects, we need to worry much less about what happens *inside* a zone and more about what happens *between* the zones. That means we need to spend time thinking about how our microservices talk to each other and ensuring that we can properly monitor the overall health of our system. From an architecture space, this is how we create our own *universal space*—by defining some specific boundaries, we highlight to our colleagues building the system those areas where changes can be made more freely without breaking some fundamental aspect of our architecture.

To look at a very simple example, in Figure 16-1 we see the `Recommendations` microservice accessing information from the `Promotions` and `Sales` microservices. As we've already covered at length, we are free to change the functionality hidden inside these three microservices without worrying about breaking the overall system—I can change whatever I want in `Sales` or `Promotions` as long as I continue to maintain the expectations that `Recommendations` has about how it will interact with these downstream microservices.

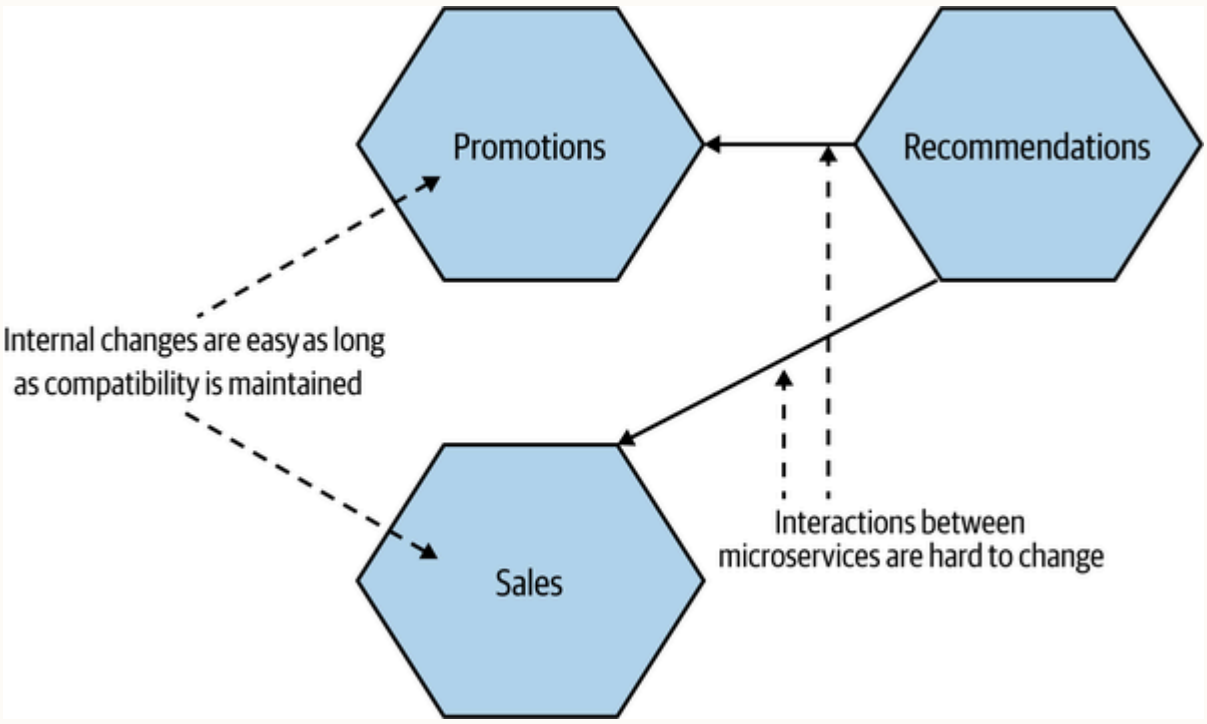


Figure 16-1. Changes inside a microservice boundary are easy to make, as long as the interactions between microservices don't change

We can create space for change at larger-scope levels as well. In Figure 16-2, we see the microservices from Figure 16-1 actually exist in a marketing zone that maps to a specific team's responsibility. We've defined an expected behavior in terms of how the marketing functionality interacts with the larger system. Inside the marketing zone, we can make any changes we like, as long as compatibility with the larger system is maintained. Coming back to the idea of understanding what things are hard to change, organizational structures often fall into this category, and as such existing team structures can help define these zones for you. Coordinating changes within a team across microservices owned by that team will be easier than changing the interactions that are exposed to other teams.

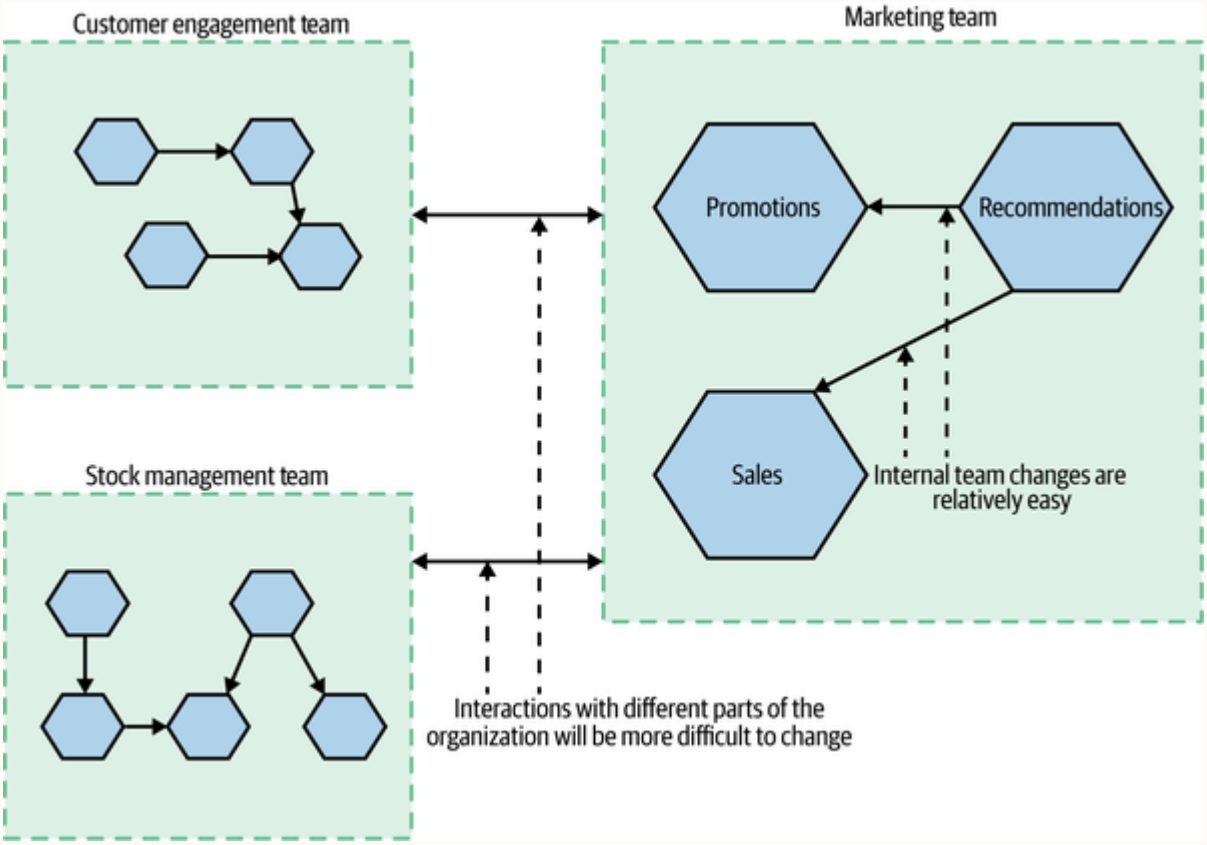


Figure 16-2. Changes within a zone are easier to make than changes between zones

This ties in nicely with the concept of a team API, which we discussed in "Small Teams, Large Organization". An architect can help facilitate creation of a team API, making sure the team's microservices and working practices fit in with the wider organization.

By defining spaces in which these changes can be made without compromising the system as a whole, we make developers' lives easier and also focus our attention on parts of the system that are harder to change. Remember the concept of information hiding that we explored in Chapter 2? As we explored there, hiding information inside a microservice boundary makes it much easier to create a stable interface for consumers. When we make changes to the microservice, it is easier to ensure we haven't broken compatibility with external consumers. Here, we can define an architecture to provide information hiding at the team level, rather than just at the microservice level. This gives us another

level of information hiding and creates a larger safe space in which a team can make local changes without breaking the wider system.

Within each microservice or larger zone, you may be OK with the team that owns that zone picking a different technology stack or data store. Other concerns may kick in here, of course. Your inclination to let teams pick the right tool for the job may be tempered by the fact that it becomes harder to hire people or move them between teams if you have 10 different technology stacks to support. Similarly, if each team picks a completely different data store, you may find yourself lacking enough experience to run any of them at scale. Netflix, for example, has mostly standardized on Cassandra as a datastore technology. Although it may not be the best fit for all of its cases, Netflix feels that the value gained by building tooling and expertise around Cassandra is more important than having to support and operate at scale multiple other platforms that may be a better fit for certain tasks. Netflix is an extreme example, where scale is likely the strongest overriding factor, but you get the idea.

Between microservices is where things can get messy, however. If one microservice decides to expose REST over HTTP, another makes use of gRPC, and a third uses Java RMI, then integration can become a nightmare, as consuming microservices have to understand and support multiple styles of interchange. This is why I try to stick to the guideline that we should "be worried about what happens between the boxes, and be liberal in what happens inside."

So a successful architecture is as much about allowing for change to suit the needs of our users as anything. But one thing people often forget is that our system doesn't just accommodate users; it also accommodates the people that actually build the software themselves. A successful architecture also helps create a nice environment in which to do our work.

A Social Construct

> *No plan survives contact with the enemy.*
>
> —Helmuth von Moltke (heavily paraphrased)

So you've thought about the vision, about the constraints, and about what you need to accomplish. You think you understand what will be hard to change, and the spaces where you need to make change possible. Now what? Well, the architecture is what happens, not what you think should happen—this is the difference between vision and real-

ity. Architects of the built environment need to work with the people constructing the building to help them understand what the vision is, but also to change the plan when reality challenges that vision. It's possible that what you think is possible fundamentally isn't. If an architect isn't embedded to some extent with the people creating the system, then they will be unable to help communicate the vision to the people doing the work, nor will the architect understand where that vision is no longer fit for purpose. The construction crew may encounter things on the ground that weren't foreseen, or perhaps a supply shortage might cause a rethink in terms of the design.

Architecture is what happens, not what is planned. If as an architect you remove yourself from the process of putting this vision into place, then you're not an architect—you're a dreamer. The architecture that will emerge may or may not bear any relationship to what you want. It will happen with or without you. Implementing an architecture requires the work of many people and a host of decisions, large and small.

As Grady Booch put it:[4]

> *In the beginning, the architecture of a software-intensive system is a statement of vision. In the end, the [a]rchitecture of every such system is a reflection of the billions upon billions of small and large, intentional and accidental design decisions made along the way.*

This means that even if you have a dedicated individual who is ultimately accountable for the architecture, there are many people responsible for putting this vision into practice. Implementing a successful architecture is going to be a team effort. Coming back to Ralph Johnson's quote from earlier, "architecture is a social construct." A great example of this comes from Comcast, which has shared its experiences of how it decentralized decision making through the use of an architecture guild.[5] Given its scale, Comcast decided to leverage experiences from industry steering groups, where collective decision making is key:

> *At Comcast we realized this problem looked very similar to the way open standards bodies work: getting multiple autonomous groups to agree on technical approaches. We designed an internal Architecture Guild explicitly modeled after a very successful standards body, the Internet Engineering Task Force (IETF) that defines many important Internet protocols.*
>
> —Jon Moore, chief software architect at Comcast Cable

Comcast's approach has a level of formality that some organizations might find onerous, but it seems to work well for the company, given its size and distribution.

Habitability

Yet another concept that comes from the built environment and has resonance in the field of software development is *habitability*. I first learned of this term from Frank Buschmann—he explained that an architect has responsibility for ensuring that the environment they create is nice to work in. If the architecture is the framing of the system, which describes how the hard-to-change things fit together, then there are also times when constraints may need to be put in place. Get this wrong, though, and working in the system can become painful and error-prone.

As Richard Gabriel, author of *Patterns of Software*,[6] explains:

> *Habitability is the characteristic of source code that enables programmers coming to the code later in its life to understand its construction and intentions and to change it comfortably and confidently.*

A modern software development ecosystem consists of more than just code, however—it extends beyond this to the technologies we use and the working practices we adopt. All too frequently I've seen developers cursing the technology they are told to use—often technology selected by people who don't ever have to make use of it. The more you make the evolution of your architecture and the selection of the tools and technology you use a collaborative process, the easier it will be for you to ensure that the end result is a habitable environment in which the people building the system feel happy and productive in their work.

If we are to ensure that the systems we create are habitable for our developers, then our architects and other decision makers need to understand the impact of their decisions. At the very least, this means spending time with the team, and ideally actually spending time coding with the team. For those of you who practice pair programming, it becomes a simple matter for an architect to join a team for a short period as one member of a pair. Participating in ensemble programming exercises could also yield significant benefits, although an architect taking part in such a group activity needs to be aware how their presence may change the dynamic of the ensemble.

Ideally, you should work on normal tasks to really understand what "normal" work is like. I cannot emphasize how important it is for the architect to actually spend time with the teams building the system! This is significantly more effective than having a call or just looking at their code. As for how often you should do this, that depends greatly on the size of the team(s) you are working with. But the key is that it should be a routine activity. If you are working with four teams, for example, perhaps make sure you spend

half a day with each team every four weeks, working with them on their delivery tasks to ensure you build awareness and improved communications with the teams you are working with.

A Principled Approach

> *Rules are for the obedience of fools and the guidance of wise men.*
>
> —Generally attributed to Douglas Bader

Making decisions in system design is all about trade-offs, and microservice architectures give us lots of trade-offs to make! When picking a data store, do we pick a platform that we have less experience with but that gives us better scaling? Is it OK for us to have two different technology stacks in our system? What about three? Some decisions can be made completely on the spot with the information available to us, and these are the easiest to make. But what about those decisions that might have to be made on incomplete information?

Framing can help here, and a great way to help frame our decision making is to define a set of principles and practices that guide it, based on goals that we are trying to achieve. Let's look at each of these aspects of framing in turn.

Strategic Goals

The role of the architect is already daunting enough, so luckily we usually don't have to also define strategic goals! Strategic goals should speak to where your company is going and to how it sees itself as best making its customers happy. These will be high-level goals and may not include technology at all. They could be defined at a company level or a division level. They might be things like "Expand into Southeast Asia to unlock new markets," or "Let the customer achieve as much as possible using self-service." The key is that they define where your organization is headed, so you need to make sure the technology is aligned to that.

If you're the person defining the company's technical vision, this may mean you'll need to spend more time with the nontechnical parts of your organization (or "the business," as they are often called). What is the driving vision for the business? And how does it change?

Principles

Principles are rules you have made in order to align what you are doing to some larger goal, and they will sometimes change. For example, if one of your strategic goals as an organization is to decrease the time to market for new features, you may define a principle that says that delivery teams have full control throughout the life cycle of their software to ship whenever they are ready, independently of any other team. If another goal is that your organization is moving to aggressively grow its offerings in other countries, you may decide to implement a principle that the entire system must be portable to allow for it to be deployed locally in order to respect sovereignty of data.

You probably don't want loads of these. Fewer than 10 principles is a good number—small enough for people to remember them, or to fit on small posters. The more principles you have, the greater the chance that they overlap or contradict each other.

Heroku's Twelve Factors is a set of design principles structured around the goal of helping you create applications that work well on the Heroku platform. These principles may also make sense in other contexts. Some of them are actually constraints based on behaviors your application needs to exhibit in order to work on Heroku. A constraint is really something that is very hard (or virtually impossible) to change, whereas principles are things we decide to choose. You may decide to explicitly call out those things that are principles versus those that are constraints to help highlight those things you really can't change. Personally, I think there can be some value in keeping them in the same list, to encourage challenging constraints every now and then and see if they really are immovable!

Practices

Our practices are how we ensure our principles are being carried out. They are a set of detailed, practical guidelines for performing tasks. They will often be technology specific and should be low level enough that any developer can understand them. Practices could include coding guidelines, the fact that all log data needs to be captured centrally, or the fact that HTTP/REST is the standard integration style. Due to their technical nature, practices will typically change more often than principles.

As with principles, sometimes practices reflect constraints within your organization. For example, if you have decided to pick Azure as your cloud platform, this will need to be reflected in your practices.

Practices should underpin your principles. A principle stating that delivery teams control the full life cycle of their systems may mean you have a practice stating that all microservices are deployed into isolated AWS accounts, providing self-service management of the resources and isolation from other teams.

Combining Principles and Practices

One person's principles are another's practices. You might decide to call the use of HTTP/REST a principle rather than a practice, for example. And that would be fine. The key point is that there is value in having overarching ideas that guide how the system evolves, and in having enough detail so that people know how to implement those ideas. For a small enough group, perhaps a single team, combining principles and practices might be OK. However, for larger organizations, where the technology and working practices may differ from place to place, you may want a different set of practices in different places, as long as they all map to a common set of principles. A .NET team, for example, might have one set of practices, and a Java team another. The principles, though, could be the same for both.

A Real-World Example

An old colleague of mine, Evan Bottcher, developed the diagram shown in Figure 16-3 in the course of working with one of his clients. The figure shows the interplay of goals, principles, and practices in a very clear format. Over the course of a couple of years, the practices on the far right will change fairly regularly, whereas the principles remain fairly static. A diagram such as this can be printed nicely on a single sheet of paper and shared, and each idea is simple enough for the average developer to remember. There is, of course, more detail behind each point here, but being able to articulate this in summary form is very useful.

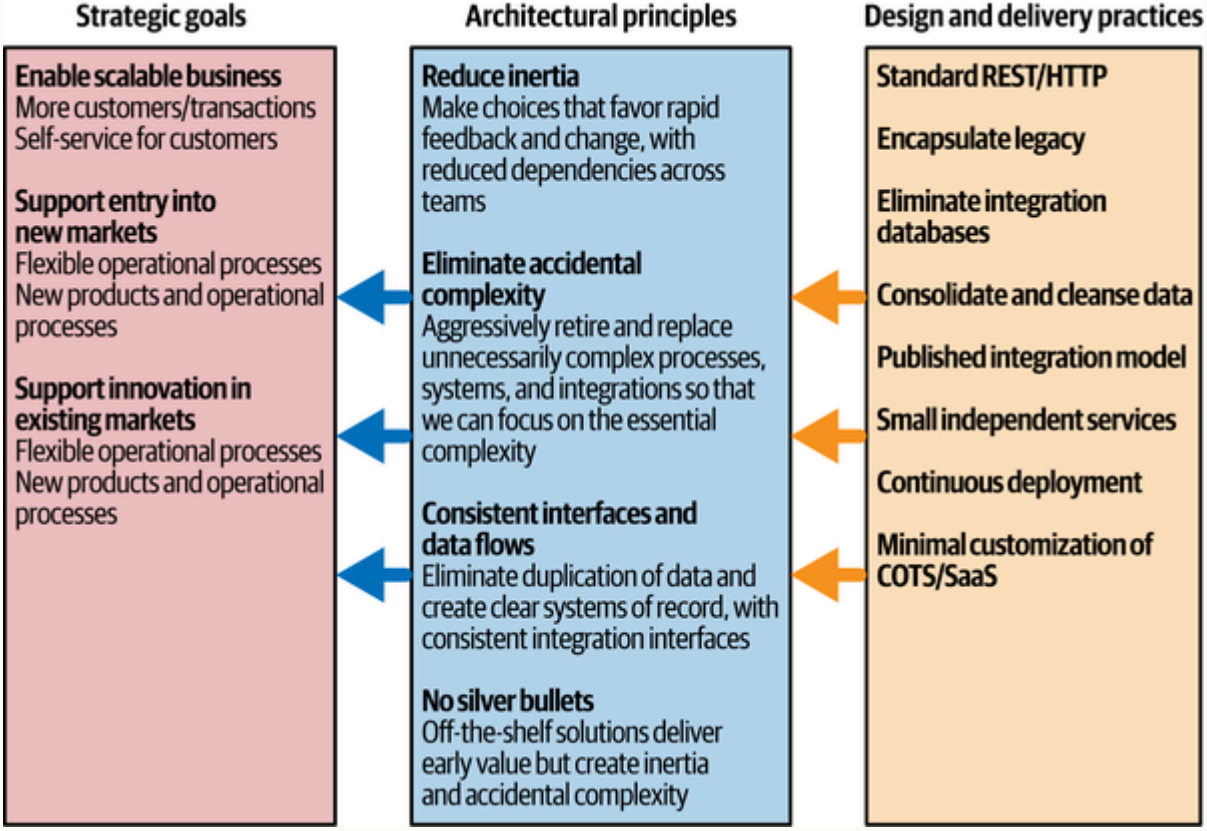


Figure 16-3. A real-world example of principles and practices

It makes sense to have documentation supporting some of these items, and even better is having working code that shows how these practices can be implemented. In "The Platform", we looked at how the creation of a common set of tools can make it easy for developers to do the right thing—ideally, the platform should make following these practices as easy as possible, and as the practices change, the platform should change accordingly.

Guiding an Evolutionary Architecture

So if our architecture is not static but is ever-changing and evolving, how do we make sure it is growing and changing in the way we want, rather than just mutating into some unmanageable giant blob of pain, suffering, and recriminations? In *Building Evolutionary Architectures*,[7] the authors outline fitness functions to help collect information about the relative "fitness" of the architecture in order to help architects decide if they need to take action. From the book:

> *Evolutionary computing includes a number of mechanisms that allow a solution to gradually emerge via small changes in each generation of the software. At each generation of the solution, the engineer assesses the current state: Is it closer to or further away from the ultimate goal? For example, when using a genetic algorithm to optimize wing design, the fitness function assess[es] wind resistance, weight, air flow, and other characteristics desirable to good wing design. Architects define a fitness function to explain what better is and to help measure when the goal is met. In software, fitness functions check that developers preserve important architectural characteristics.*

The idea of a fitness function is that it is used to understand the current state of some important property, such that if that property changes outside of some allowable bounds, then the change needs to be looked into. Typically, fitness functions will be used to ensure that the architecture is being built to follow the principles and constraints that have been laid down.

To borrow an example from *Building Evolutionary Architectures*, consider the requirement that the response from a given service must be received in 100 ms or less. You could implement a fitness function to collect performance data from this service, perhaps either in a performance test environment or from a real-world running system to ensure that the actual behavior of the system meets the requirements. *Building Evolutionary Architectures* goes into a lot more detail on this topic, and I thoroughly recommend it if you want to explore this concept further.

Fitness functions for architecture can come in many shapes and forms. The fundamental concept, though, is that you collect real-world data to understand whether or not your architecture is achieving "fitness" against that criteria. This could relate to system performance, code coupling, cycle time, or a host of other aspects. These fitness functions act as another source of information to help an architect understand where they might need to get involved. Please note, however, that for me, fitness functions work best when combined with close collaboration with the people building the system. Fitness functions should be a useful way to help you understand if the architecture is moving in the right direction, but they don't replace the need to actually speak with people on the ground. In fact, I'd suggest that defining the right fitness functions will require close collaboration.

Architecture in a Stream-Aligned Organization

In Chapter 15, we looked at how modern software delivery organizations are shifting toward a more stream-aligned model in which autonomous independent teams focus on the end-to-end delivery of functionality, with their priorities being product driven. We also talked about cross-cutting teams—*enabling teams* that support stream-aligned teams. Where does the architect fit into this world? Well, sometimes the scope of a stream-aligned team is complex enough to require a dedicated architect (here again we often see a blurring of the lines between the traditional technical lead and architect roles). In many cases, though, architects are asked to work across multiple teams.

Many of the responsibilities of the architect can be seen as enabling responsibilities—clearly communicating technical vision, understanding challenges as they emerge, and helping adapt the technical vision accordingly. The architect helps connect people, keeping an eye on the bigger picture and helping teams understand how what they are doing fits into the greater whole. This fits neatly into the idea of an architect being part of an enabling team, as we see in Figure 16-4. Such an enabling team could consist of a mix of people—perhaps folks who are dedicated to the team full-time, and others who pitch in to help from time to time.

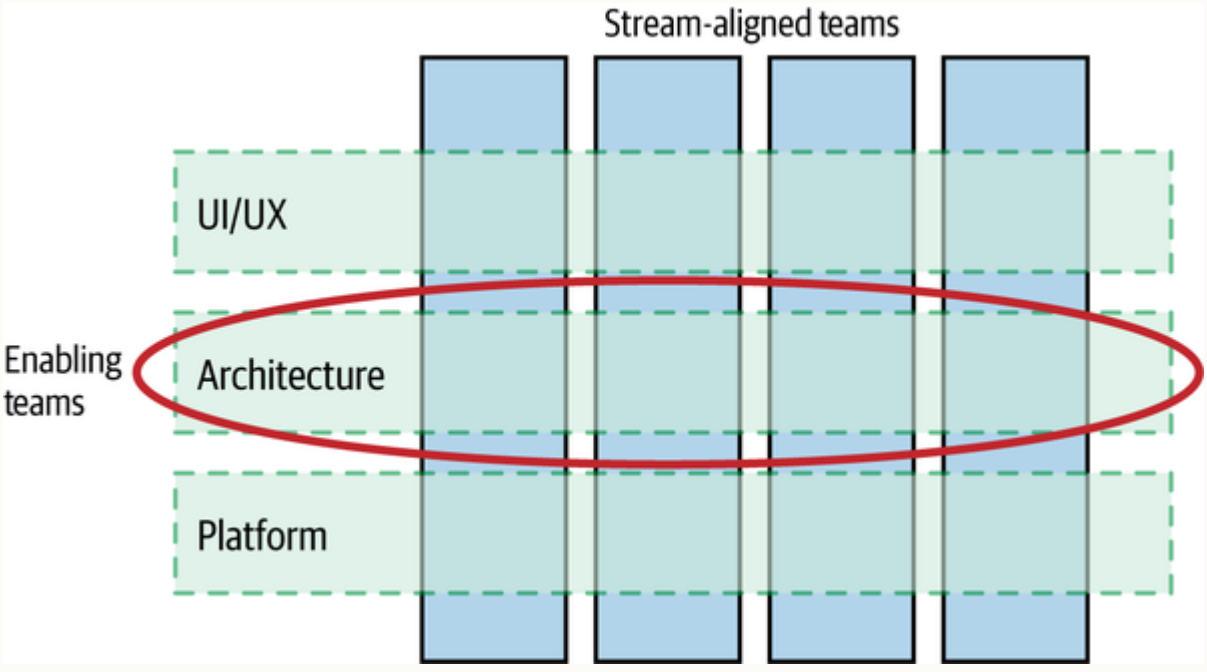


Figure 16-4. An architecture function as an enabling team

A model I greatly favor is to have a small number of dedicated architects in this team (perhaps just one or two people in many cases), but having this team augmented over time with technologists from each delivery team—the technical leads of each team at a minimum. The architect is responsible for making sure the group works. This distributes the work and ensures that there is a higher level of buy-in. It also ensures that informa-

tion flows freely from the teams into the group, and as a result, the decision making is much more sensible and informed.

Sometimes the group may make decisions that the architect disagrees with. At this point, what is the architect to do? Having been in this position before, I can tell you this is one of the most challenging situations to face. Often I take the approach that I should go with the group decision. I take the view that I've done my best to convince people, but ultimately I wasn't convincing enough. The group is often much wiser than the individual, and I've been proven wrong more than once! And imagine how disempowering it can be for a group to have been given space to come up with a decision and then ultimately be ignored. But sometimes I have overruled the group. But why, and when? How do you draw the lines?

Think about teaching a child to ride a bike. You can't ride it for them. You watch them wobble, but if you step in every time it looks like they might fall off, then they'll never learn, and in any case they fall off far less than you think they will! But if you see them about to veer into traffic or into a nearby duck pond, then you have to step in. Of course, I've frequently been proven wrong in such situations—I've let the team go off and do something that I felt was wrong, and what they did worked! Likewise, as an architect, you need to have a firm grasp of when, figuratively, your team is steering into a duck pond. You also need to be aware that even if you know you are right and overrule the team, this can undermine your position and also make the team feel that they don't have a say. Sometimes the right thing is to go along with a decision you don't agree with. Knowing when to do this and when not to is tough but sometimes vital.

Where things get interesting, as we'll discuss shortly, is when an architect also has to get involved in governance activities. This can cause some confusion about the role of any cross-cutting architecture team. What happens when one team diverges from the technical strategy? Is that OK? Perhaps it's a sensible exception, but it might also cause more fundamental issues. A short-term decision made in the name of expediency might compromise bigger changes that are trying to be made. Imagine that the architecture group is trying to help shift the organization away from the use of centralized data due to the coupling and operational issues it causes, but one of the teams decides to just throw some new data into a shared database, as it is under pressure to deliver quickly. What happens then?

In my experience, this all comes down to good, clear communication and an understanding of responsibilities. If I saw a product owner making decisions that I felt were going to undermine some sort of cross-cutting activities I was working toward, I'd go and have a chat with them. Perhaps the answer is that the short-term decision is right (and arguably

this ends up being some sort of technical debt that we have consciously taken onboard). In other cases, perhaps the product owner is able to change what they are planning to help work with the overall strategy. In the worst cases, the issue might need to be escalated.

At REA, the online real-estate company I've talked about in a few earlier chapters, product owners would occasionally make decisions to prioritize work in such a way that it caused technical debt to build up, leading to subsequent problems. The issue was that the product owners were primarily held to account in regard to their ability to deliver features and make customers happy—whereas often the issues around technical debt were laid at the feet of the technical leaders. A shift was made to make the product owners also responsible for aspects of the software that were technical in nature—this meant that they had to take a more active role in understanding the more technical aspects of the system (security or performance, for example) and work more collaboratively with the technical experts in terms of prioritizing work to be done. The act of making nontechnical product owners more accountable for prioritization around technical activities is nontrivial, but it is absolutely worth it in my experience.

Building a Team

Being the main point person for the technical vision of your system and ensuring that you're executing on this vision isn't just about making sure the right technology decisions get made. It's the people you work with who will be doing the work. It is the role of any technical leader to help these people grow—to help them be part of creating that vision—and to ensure that they can be active participants in shaping and implementing the vision too.

Helping the people around you with their own career growth can take many forms, most of which are outside the scope of this book. There is one aspect, though, in which a microservice architecture is especially relevant. With larger, monolithic systems, there are fewer opportunities for people to step up and "own" something. With microservices, on the other hand, we have multiple autonomous codebases that will have their own independent life cycles. Helping people step up by having them take ownership of individual microservices before accepting more responsibility can be a great way to help them achieve their own career goals, and at the same time it lightens the load on whomever is in charge!

I am a strong believer that great software comes from great people. If you worry only about the technology side of the equation, you're missing way more than half the pic-

ture.

The Required Standard

When you're working through your practices and thinking about the trade-offs you need to make, one of the most important balances to find is how much variability to allow in your system. One of the key ways to identify what should be constant from microservice to microservice is to define what a well-behaved, good microservice looks like. What is a "good citizen" microservice in your system? What capabilities does it need to have to ensure that your system is manageable, and that one bad microservice doesn't bring down the whole system? As with people, what a "good citizen" microservice is in one context does not reflect what it looks like somewhere else. Nonetheless, there are some common characteristics of well-behaved microservices that I think are fairly important to observe. These are the few key areas in which allowing too much divergence can result in a pretty horrid time. As Ben Christensen from Facebook puts it, when you think about the bigger picture, "it needs to be a cohesive system made of many small parts with autonomous life cycles but all coming together." So you need to find a balance in which you optimize for the autonomy of individual microservices without losing sight of the bigger picture. Defining clear attributes that each microservice should have is one way of being clear as to where that balance sits. Let's touch on some of those attributes.

Monitoring

It is essential that we are able to draw up coherent, cross-service views of our system health. This has to be a system-wide view, not a microservice-specific view. As we discussed in Chapter 10, knowing the health of an individual microservice is useful, but often only when you're trying to diagnose a wider problem or understand a larger trend. To make this as easy as possible, I would suggest ensuring that all microservices emit health-related and general-monitoring-related metrics in the same way.

You might choose to adopt a push mechanism, where each microservice needs to push this data into a central location. Whatever you pick, try to keep it standardized. Make the technology inside the box opaque, and don't require that your monitoring systems change in order to support it. Logging falls into the same category here: we need it in one place.

Interfaces

Picking a small number of defined interface technologies helps integrate new consumers. Having one standard is good. Two isn't too bad, either. Having twenty different styles of integration is not good. This isn't just about picking the technology and the protocol. If you pick HTTP/REST, for example, will you use verbs or nouns? How will you handle pagination of resources? How will you handle versioning of endpoints?

Architectural Safety

We cannot afford for one badly behaved microservice to ruin the party for everyone. We have to ensure that our microservices shield themselves accordingly from unhealthy, downstream calls. The more microservices we have that do not properly handle the potential failure of downstream calls, the more fragile our systems will be. This might mean, for example, that you want to mandate certain practices around inter-service communication, such as requiring the use of circuit breakers (a topic we explored in "Stability Patterns").

Playing by the rules is important when it comes to response codes, too. If your circuit breakers rely on HTTP codes, and one microservice decides to send back 2XX codes for errors or confuses 4XX codes with 5XX codes, then these safety measures can fall apart. Similar concerns would apply even if you're not using HTTP; we need to know the difference between a request that was OK and processed correctly, a request that was bad and thus prevented the microservice from doing anything with it, and a request that might be OK but we can't tell because the server was down. Knowing this is key to ensuring we can fail fast and track down issues. If our microservices play fast and loose with these rules, we end up with a more vulnerable system.

Governance and the Paved Road

Part of what architects need to handle is governance. What do I mean by *governance*? It turns out the COBIT (Control Objectives for Information Technologies) framework has a pretty good definition:[8]

> *Governance ensures that enterprise objectives are achieved by evaluating stakeholder needs, conditions and options; setting direction through prioritization and decision making; and monitoring performance, compliance and progress against agreed-on direction and objectives.*

In a nutshell, we can consider governance as agreeing how things should be done, making sure people know how things should be done, and making sure things are done that way. In some environments, governance just happens informally, as part of normal software development activities. In other environments, especially within larger organizations, this might need to be a more concrete function.

Governance can apply to multiple things in the forum of IT. We want to focus on the aspect of technical governance, something I feel is the job of the architect. If one of the architect's jobs is ensuring there is a technical vision, then governance is about ensuring that what we are building matches this vision, and evolving the vision if needed.

Fundamentally, governance should be a group activity. A properly functioning governance group can work together to share the work and shape the vision. It could be an informal chat with a small enough team, or a more structured regular meeting with formal group membership for a larger scope. This is where I think the principles we covered earlier should be discussed and changed as required. If a formal group is required, this group needs to consist predominantly of people who are executing the work being governed. This group should also be responsible for tracking and managing technical risks.

Getting together and agreeing on how things can be done is a good idea. But spending time making sure people are following these guidelines is less fun, as is placing a burden on developers to implement all these standard things you expect each microservice to do. I am a great believer in making it easy to do the right thing—and as we discussed in Chapter 15, the paved road is a really useful concept here. The architect has a role to clearly articulate the vision—where you are going—and to make it easy to get there. As such, they should be involved in helping shape the requirements of whatever paved road you build. For many, the platform will be the biggest example of this—the architect ends up being an important stakeholder for the platform team.

We've already looked at the role of the platform in some depth, so let's look at a couple of other techniques we can use to make it as easy as possible for people to do the right thing.

Exemplars

Written documentation is good and useful. I clearly see the value in it; after all, I've written this book. But developers also like code—code they can run and explore. If you have a set of standards or best practices you would like to encourage, then having exemplars you can point people to is useful. The idea is that people can't go far wrong just by imitating some of the better parts of your system.

Ideally, these should be real-world microservices running in your system that get things right, rather than isolated microservices that are implemented merely as "perfect examples." By ensuring your exemplars are actually being used, you ensure that all the principles you have actually make sense.

Tailored Microservice Template

Wouldn't it be great if you could make it really easy for all developers to follow most of the guidelines you have with very little work? What if, out of the box, the developers had most of the code in place to implement the core attributes that each microservice needs?

Many frameworks exist for different programming languages that attempt to give you the building blocks for your own microservice template. Spring Boot is probably the most successful example of such a framework for the JVM. The core Spring Boot framework is fairly light, but you can then decide to pull together a set of libraries to provide features like checking health, serving HTTP, or exposing metrics. So right out of the box, you have a simple "Hello World" microservice that can be launched from the command line.

Many people then take these frameworks and standardize this setup for their company. For example, when spinning up a new microservice, they may script things so that they get a Spring Boot template with the core libraries their organization uses already wired in; it might already pull in the libraries to handle circuit breakers and be configured to handle JWT authentication for inbound calls. Normally, such an automated template creation would create a matching build pipeline as well.

Caution warranted

The selection and configuration of these tailored microservices templates is commonly a task for the platform team. They might, for example, provide a template for each supported language, ensuring that when using the template the resulting microservices work well with the platform itself. This can cause challenges, however.

I have seen many a team's morale and productivity destroyed by having a mandated framework thrust upon it. In a drive to improve code reuse, more and more work is placed into a centralized framework until it becomes an overwhelming monstrosity. If you decide to use a tailored microservice template, think very carefully about what its job is. Ideally, its use should be purely optional, but if you are going to be more forceful in its adoption, you need to understand that ease of use for the developers has to be a

prime guiding force. Allowing the developers who use the template to recommend and even contribute changes to the framework, perhaps as part of an internal open source model, can help greatly here.

As we discussed in "DRY and the Perils of Code Reuse in a Microservice World", we have to be aware of the perils of shared code. In our desire to create reusable code, we can introduce sources of coupling between microservices. At least one organization I spoke to is so worried about this that it actually copies its microservice template code manually into each microservice. This means that an upgrade to the core microservice template takes longer to be applied across its system, but this is less concerning to the organization than the danger of coupling. Other teams I've spoken to have simply treated the microservice template as a shared binary dependency, although they have to be very diligent in not letting the tendency for DRY (don't repeat yourself) result in an overly coupled system!

The Paved Road at Scale

The use of in-house internal microservice templates and frameworks is often found in organizations that have large numbers of microservices. Netflix and Monzo are two such organizations. Each has decided to standardize on its technology stack to some degree (the JVM in the case of Netflix, Go in terms of Monzo), allowing it to speed up the creation of a new microservice with standard, expected behavior by using a common set of tools. With a more divergent technology stack, having a standard microservice template for your own needs becomes more difficult.

If you were to embrace multiple disparate technology stacks, you'd need a matching microservice template for each. This could be a way you subtly constrain language choices in your teams, though. If the in-house microservice template supports only the JVM, then people may be discouraged from picking alternative stacks if they have to do lots more work themselves. Netflix, for example, is especially concerned with aspects like fault tolerance to ensure that the outage of one part of its system cannot take everything down. To handle this, a large amount of work has been done to ensure that there are client libraries on the JVM to provide teams with the tools they need to keep their microservice well behaved. Introducing a new technology stack would mean having to reproduce all this effort. The main concern for Netflix is less about the duplicated effort and more about the fact that it is so easy to get this wrong. The risk of a microservice getting newly implemented fault tolerance wrong is high if it could impact more of the system. Netflix mitigates this by using "sidecar services," which communicate locally with a JVM that is using the appropriate libraries.

Service meshes have given us another potential way to offload common behavior. Some functionality that was commonly seen as an internal microservice's responsibility can now be pushed to a microservice mesh. This can ensure more consistency of behavior across microservices written in different programming languages and also reduce the responsibilities of these microservice templates.

Technical Debt

We are often put in situations in which we cannot follow through to the letter on our technical vision. Often, we need to make a choice to cut a few corners to get some urgent features out. This is just one more trade-off that we'll find ourselves having to make. Our technical vision exists for a reason. If we deviate from this reason, it might have a short-term benefit but a long-term cost. A concept that helps us understand this trade-off is technical debt. When we accrue technical debt, just like debt in the real world it has an ongoing cost and is something we want to pay down.

Sometimes technical debt isn't just something we cause by taking shortcuts. What happens if our vision for the system changes, but not all of our system matches? In this situation, too, we have created new sources of technical debt.

The architect's job is to look at the bigger picture and understand this balance. Having some view as to the level of debt and where to get involved is important. Depending on your organization, you might be able to provide gentle guidance, but have the teams themselves decide how to track and pay down the debt. For other organizations, you may need to be more structured, perhaps maintaining a debt log that is reviewed regularly.

Exception Handling

So our principles and practices guide how our systems should be built. But what happens when our system deviates from that? Sometimes we make a decision that is just an exception to the rule. In these cases, it might be worth capturing such a decision in a log somewhere for future reference. If enough *exceptions* are found, it may eventually make sense to change the applicable principle or practice to reflect a new understanding of the world. For example, we might have a practice that states that we will always use MySQL for data storage. But then we see compelling reasons to use Cassandra for highly scalable storage, at which point we change our practice to say, "Use MySQL for most

storage requirements, unless you expect large growth in volumes, in which case use Cassandra."

It's worth reiterating, though, that every organization is different. I've worked with some companies at which the development teams have a high degree of trust and autonomy, and the principles are lightweight (and the need for overt exception handling is greatly reduced, if not eliminated). In more structured organizations in which developers have less freedom, tracking exceptions may be vital to ensuring that the rules in place properly reflect the challenges people are facing. With all that said, I am a fan of microservices as a way of optimizing for autonomy of teams, giving them as much freedom as possible to solve the problem at hand. If you are working in an organization that places lots of restrictions on how developers can do their work, then microservices may not be for you.

Summary

To summarize this chapter, here are what I see as the core responsibilities of the evolutionary architect:

Vision

: Ensure there is a clearly communicated technical vision for the system that will help it meet the requirements of your customers and organization.

Empathy

: Understand the impact of your decisions on your customers and colleagues.

Collaboration

: Engage with as many of your peers and colleagues as possible to help define, refine, and execute the vision

Adaptability

: Make sure that the technical vision changes as required by your customers or organization.

Autonomy

: Find the right balance between standardizing and enabling autonomy for your teams.

Governance

> Ensure that the system being implemented fits the technical vision, and make sure it is easy for people to do the right thing.

The evolutionary architect is one who understands that demonstrating these core responsibilities is a constant balancing act. Forces are always pushing us one way or another, and understanding where to push back or where to go with the flow is often something that comes only with experience. But the worst reaction to all these forces that push us toward change is to become more rigid or fixed in our thinking.

While much of the advice in this chapter can apply to any systems architect, microservices give us many more decisions to make. Therefore, being better able to balance all of these trade-offs is essential. If you want to explore this topic in more depth, I can recommend the already quoted *Building Evolutionary Architectures*, as well as Gregor Hohpe's *The Software Architect Elevator*,[9] which helps architects understand how they can bridge the gap between high-level strategic thinking and on-the-ground delivery.

We're almost at the end of the book, and we've covered a lot of ground. In the Afterword, we'll now summarize what we've learned.

1 For several reasons, not the least of which is that I have a degree in software engineering…

2 This is from an email exchange on the extreme programming mailing list, which Martin Fowler then shared in his article "Who Needs an Architect?".

3 Elevators, for my North American readers.

4 Grady Booch (@Grady_Booch), Twitter, September 4, 2020, 5:12 a.m., *https://oreil.ly/ZgPRZ*.

5 Jon Moore, "Architecture with 800 of My Closest Friends: The Evolution of Comcast's Architecture Guild," InfoQ, May 14, 2019, *https://oreil.ly/aIvbi*.

6 Richard Gabriel, *Patterns of Software: Tales from the Software Community* (New York: Oxford University Press, 1996).

7 Neal Ford, Rebecca Parsons, and Patrick Kua, *Building Evolutionary Architectures* (Sebastopol: O'Reilly, 2017).

8 *COBIT 5: A Business Framework for the Governance and Management of Enterprise IT* (Rolling Meadows, IL: ISACA, 2012).

9 Gregor Hohpe, *The Software Architect Elevator* (Sebastopol: O'Reilly, 2020).

Previous chapter
< 15. Organizational Structures

Next chapter
Afterword: Bringing It All Together >